Universidade do Minho

Ano Letivo: 2025/26

Bases de Dados

A Liguagem SQL (continuação)



Sumário

1 Controlo de Dados (DCL)
- *Gestão de Utilizadores*
- *Atribuição e Remoção de privilégios*
- *Grupos de privilégios*

2 Vistas, Índices, Prepared Statements, Procedimentos

Bibliografia:

- Connolly, T., Begg, C., Database Systems, A Practical Approach to Design, Implementation, and Management , Addison-Wesley, 4a Edição, 2004. **(Chapter 18)**

- Belo, O., "Bases de Dados Relacionais: Implementação com MySQL", FCA – Editora de Informática, 376p, Set 2021. ISBN: 978-972-722-921-5. **(Chapter 2)**

Base de Dados

Sakila

A **Sakila** é uma base de dados de exemplo desenvolvida pela MySQL e vai ser usada nos exemplos das próximas aula.

Representa o funcionamento de uma **loja de aluguer de filmes.**

O que inclui?

Informação sobre:

- filmes
- clientes
- lojas
- pagamentos
- alugueres

Tutorial:

https://dev.mysql.com/doc/sakila/en/sakila-introduction.html

Instalação:

https://dev.mysql.com/doc/sakila/en/sakila-installation.html

A Linguagem SQL

Gestão de Utilizadores

- A gestão de utilizadores num sistema de bases de dados é feita através de instruções de **definição de dados (DDL)**

- Para que um utilizador possa aceder à base de dados, é necessário:
  - ter um **perfil criado no sistema**
  - possuir **permissões (privilégios)** que definam o que pode ou não fazer

- A criação de utilizadores é realizada com a instrução **CREATE USER**

- Para modificar ou eliminar utilizadores utilizam-se, respetivamente:
  - **ALTER USER**
  - **DROP USER**

A Linguagem SQL

Gestão de Utilizadores

- É possível criar utilizadores com acesso local ou remoto a uma base de dados
- Cada utilizador é identificado por: **nome, origem (host)** e **palavra-passe**

Criar utilizador local

```
CREATE USER 'cris'@'localhost'
IDENTIFIED BY 'passCris'
```

O utilizador só pode aceder a partir do próprio sistema

Criar utilizador remoto

```
CREATE USER 'gloria'@'11.0.1.1'
IDENTIFIED BY 'passGloria';
```

O utilizador pode aceder a partir do IP especificado

Alterar palavra-passe

```
ALTER USER 'cris'@'localhost'
IDENTIFIED BY 'newCris';
```

A Linguagem SQL

Consulta de Utilizadores

A consulta de utilizadores permite obter informação sobre os acessos ao sistema.

Listar todos os utilizadores:

```
SELECT * FROM mysql.user;
```

Consultar nome, host e número máximo de conexões:

```
SELECT User, Host, max_connections
FROM mysql.user
ORDER BY User;
```

Consulta da informação relativa a um conjunto de utilizadores no sistema local:

```
SELECT User, Host
FROM mysql.user
WHERE USER IN ('james','cris')
      AND Host = 'localhost';
```

Estas consultas permitem analisar quem tem acesso ao sistema e de que forma.

A Linguagem SQL

Remoção de Utilizadores

- A remoção de utilizadores permite **eliminar acessos ao sistema de base de dados**
- É utilizada quando um utilizador deixa de necessitar de acesso ou por razões de segurança

```
DROP USER 'james'@'localhost’;
```

Aspetos importantes:

A operação é **irreversível**
O utilizador perde **todas as permissões** associadas
Deve ser usada com cuidado para evitar perda de acessos necessários

A Linguagem SQL

Atribuição de Privilégios

- A atribuição de privilégios permite **controlar o acesso dos utilizadores** aos objetos da base de dados
- Define **o que cada utilizador pode fazer** (ler, inserir, atualizar, apagar, etc.)
- Privilégios mais comuns em SQL:
  - **ALL PRIVILEGES** – Concede todos os privilégios ao utilizador
  - **CREATE** – Permite criar bases de dados e tabelas
  - **DROP** – Permite eliminar bases de dados e tabelas
  - **DELETE** – Permite remover registos de uma tabela
  - **INSERT** – Permite inserir novos registos
  - **SELECT** – Permite consultar dados
  - **UPDATE** – Permite alterar registos existentes

Atribuir privilégios: GRANT
Remover privilégios: REVOKE

A Linguagem SQL

Atribuição de Privilégios

Os privilégios podem ser atribuídos a diferentes níveis:

- Todo o sistema
- Uma base de dados
- Uma tabela

Exemplos:

```
GRANT ALL PRIVILEGES ON *.* TO 'gloria'@'localhost';
```

todos os tipos de privilégios, sobre todas as bases de dados e todas as suas tabelas

```
GRANT ALL PRIVILEGES ON sakila.* TO 'james'@'localhost';
```

todos os tipos de privilégios, sobre a base de dados sakila e todas as suas tabelas

```
GRANT ALL PRIVILEGES ON sakila.customer TO 'james'@'localhost';
```

todos os tipos de privilégios, sobre a base de dados sakila apenas para a tabela customer

Aplicar alterações:

```
FLUSH PRIVILEGES;
```

força o MySQL a recarregar as permissões

A Linguagem SQL

Atribuição de Privilégios

Exemplos:

```
GRANT SELECT, INSERT ON sakila.customer TO 'aluno'@'localhost';
```
permite ao utilizador consultar dados (SELECT) e inserir novos registos (INSERT) na tabela customer, mas não pode alterar nem apagar dados.

```
GRANT UPDATE (email) ON sakila.customer TO 'support'@'localhost';
```
permite ao utilizador atualizar apenas a coluna email da tabela customer, sem acesso às restantes colunas.

```
GRANT ALL PRIVILEGES ON sakila.* TO 'admin'@'localhost' WITH GRANT OPTION;
```
dá ao utilizador controlo total sobre toda a base de dados Sakila e permite ainda que ele atribua permissões a outros utilizadores.

```
GRANT EXECUTE ON PROCEDURE sakila.film_in_stock TO 'analista'@'localhost';
```
permite ao utilizador executar a stored procedure film_in_stock, mas não permite alterar a sua definição.

A Linguagem SQL

Limitação das atuações das contas

Queries por hora
Limita o número de consultas executadas

```
GRANT USAGE ON *.* TO 'aluno'@'localhost'
WITH MAX_QUERIES_PER_HOUR 100;
```

Modificações por hora
Limita INSERT, UPDATE e DELETE

```
GRANT USAGE ON *.* TO 'aluno'@'localhost'
WITH MAX_UPDATES_PER_HOUR 50;
```

Ligações por hora
Limita o número de conexões ao servidor

```
GRANT USAGE ON *.* TO 'aluno'@'localhost'
WITH MAX_CONNECTIONS_PER_HOUR 20;
```

Ligações simultâneas
Limita conexões ativas ao mesmo tempo

```
GRANT USAGE ON *.* TO 'aluno'@'localhost'
WITH MAX_USER_CONNECTIONS 5;
```

A Linguagem SQL

Grupos

Roles (ou grupos) são conjuntos de permissões que podem ser atribuídos a vários utilizadores.

Para que servem?
Permitem gerir permissões de forma mais simples e organizada, evitando atribuir permissões individualmente a cada utilizador.

Vantagens

- Mais fácil administrar utilizadores
- Reduz erros de permissões
- Permite reutilizar configurações de acesso

A criação de um grupo faz através da instrução **CREATE ROLE**.

A Linguagem SQL

Grupos

- **Criar um role**

```
CREATE ROLE 'analista’;
```

- **Dar permissões ao role**

```
GRANT SELECT ON sakila.* TO 'analista’;
```

- **Atribuir role a utilizador**

```
GRANT 'analista' TO 'aluno'@'localhost’;
```

- **Ativar role por defeito**

```
SET DEFAULT ROLE 'analista' TO 'aluno'@'localhost’;
```

- **Remover role**

```
DROP ROLE 'analista’;
```

A Linguagem SQL

Consulta de privilégios

Ver privilégios de um utilizador

`SHOW GRANTS FOR 'aluno'@'localhost';` -> Mostra todas as permissões atribuídas ao utilizador

Ver utilizador atual

`SELECT CURRENT_USER();` -> Mostra o utilizador ligado ao sistema

Ver roles atribuídos

`SHOW GRANTS FOR 'aluno'@'localhost';` -> Também mostra os roles associados ao utilizador

Ver roles ativos na sessão

`SELECT CURRENT_ROLE();` -> Mostra quais os roles ativos no momento

A Linguagem SQL

Vistas

- Uma view é uma tabela virtual baseada no resultado de uma consulta SQL.
- Não armazena dados fisicamente, executa a query sempre que é chamada.

Sintaxe Básica

```
CREATE VIEW nome_da_view AS
SELECT coluna1, coluna2 FROM tabela WHERE condição;
```

Para usar:

```
SELECT * FROM nome_da_view;
```

A Linguagem SQL

Vistas

Para que servem?

- **Simplificar queries complexas** — esconde JOINs e lógica repetitiva
- **Segurança** — expõe apenas as colunas/linhas necessárias ao utilizador
- **Reutilização** — uma query definida uma vez, usada em vários sítios
- **Abstração** — o código da aplicação não depende da estrutura real das tabelas

A Linguagem SQL

Vistas

Exemplo 1 — Lista de filmes disponíveis para aluguer:

```
CREATE VIEW v_filmes_disponiveis AS
SELECT title, rental_rate, rating
FROM film
WHERE rental_duration <= 5;
```

Agora basta fazer:

```
SELECT * FROM v_filmes_disponiveis;
```

Exemplo 2 — Clientes ativos:

```
CREATE VIEW v_clientes_ativos AS
SELECT first_name, last_name, email
FROM customer  WHERE active = 1;

SELECT * FROM v_clientes_ativos;
```

A Linguagem SQL

Vistas

Exemplo

Resumo dos alugueres por cliente:

```
CREATE VIEW v_resumo_clientes AS
SELECT c.customer_id, c.first_name, c.last_name,
     COUNT(r.rental_id) AS total_alugueres,
     SUM(p.amount) AS total_gasto
FROM customer c
LEFT JOIN rental r ON c.customer_id = r.customer_id
LEFT JOIN payment p ON c.customer_id = p.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name;
```

Agora basta fazer:

```
SELECT * FROM v_resumo_clientes WHERE total_gasto > 50;
```

A Linguagem SQL

Vistas

Exemplo

```
-- Alterar — adicionar coluna de duração
CREATE OR REPLACE VIEW v_resumo_clientes AS
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    COUNT(r.rental_id) AS total_alugueres,
    SUM(p.amount)      AS total_gasto,
    AVG(f.length)      AS duracao_media_filmes
FROM customer c
LEFT JOIN rental r    ON c.customer_id   = r.customer_id
LEFT JOIN payment p   ON c.customer_id   = p.customer_id
LEFT JOIN inventory i ON r.inventory_id  = i.inventory_id
LEFT JOIN film f      ON i.film_id       = f.film_id
GROUP BY c.customer_id, c.first_name, c.last_name;

-- Eliminar
DROP VIEW v_resumo_clientes;
```

A Linguagem SQL

Vistas

Views como Mecanismo de Segurança

As views podem ser usadas para **controlar e restringir o acesso aos dados**, impedindo que os utilizadores acedam diretamente às tabelas base.

Exemplo — ocultar dados sensíveis de clientes:

```
CREATE VIEW v_cliente_publico AS
SELECT customer_id, first_name, last_name, email FROM customer;
-- colunas internas ficam ocultas

GRANT SELECT ON v_cliente_publico TO utilizador_atendimento;
REVOKE SELECT ON customer FROM utilizador_atendimento;
```

A Linguagem SQL

Vistas

Views como Mecanismo de Segurança

As views podem ser usadas para **controlar e restringir o acesso aos dados**, impedindo que os utilizadores acedam diretamente às tabelas base.

Exemplo — ocultar dados sensíveis de clientes:

```
CREATE VIEW v_cliente_publico AS SELECT customer_id, first_name, last_name,
email FROM customer; -- colunas internas ficam ocultas

GRANT SELECT ON v_cliente_publico TO utilizador_atendimento; REVOKE SELECT
ON customer FROM utilizador_atendimento;
```

A Linguagem SQL

Índices

- Um índice é uma estrutura auxiliar que acelera a pesquisa de dados numa tabela, evitando percorrer todos os registos linha a linha.
- É como o índice remissivo de um livro: em vez de ler o livro todo, vais diretamente à página certa.

Criar índice simples

```
CREATE INDEX idx_nome ON tabela (coluna);
```

Criar índice único

```
CREATE UNIQUE INDEX idx_nome ON tabela (coluna);
```

Eliminar índice

```
DROP INDEX idx_nome ON tabela;
```

A Linguagem SQL

Índices

Para que servem?

Acelerar pesquisas — queries com WHERE, JOIN, ORDER BY ficam mais rápidas

Garantir unicidade — índices UNIQUE impedem valores duplicados

Melhorar JOINs — colunas usadas em JOIN beneficiam muito de índices

Desvantagem: ocupam espaço em disco e **abrandam** INSERT, UPDATE e DELETE porque o índice também precisa de ser atualizado.

A Linguagem SQL

Índices

Tipos

Tipo	Descrição
INDEX	Índice simples, permite duplicados
UNIQUE	Não permite valores duplicados
PRIMARY KEY	Índice único criado automaticamente
FULLTEXT	Pesquisa de texto livre
COMPOSITE	Índice sobre várias colunas

A Linguagem SQL

Índices

Exemplo

Pesquisa por apelido de cliente → sem índice, percorre toda a tabela:

```
SELECT * FROM customer WHERE last_name = 'SMITH';
```

Criar índice para acelerar:

```
CREATE INDEX idx_customer_lastname ON customer (last_name);
```

Agora a mesma query usa o índice e é muito mais rápida.

A Linguagem SQL

Índices

Índice Composto

Usado quando se filtra frequentemente por mais do que uma coluna:

```
CREATE INDEX idx_customer_nome ON customer (last_name, first_name);

-- Beneficia do índice
 SELECT * FROM customer WHERE last_name = 'SMITH' AND first_name = 'JOHN';

-- Também beneficia (usa a primeira coluna do índice)
SELECT * FROM customer WHERE last_name = 'SMITH';

-- NÃO beneficia (salta a primeira coluna)
SELECT * FROM customer WHERE first_name = 'JOHN';
```

A Linguagem SQL

Índices

Índice UNIQUE

```
CREATE UNIQUE INDEX idx_email ON customer (email);
```

Garante que não existem dois clientes com o mesmo email → qualquer tentativa de inserir um duplicado resulta em erro.

Ver Índices Existentes

```
-- Ver índices de uma tabela
SHOW INDEX FROM customer;
```

A Linguagem SQL

Índices

Quando usar índices?

Usar quando:

- A coluna aparece frequentemente em WHERE, JOIN ou ORDER BY
- A tabela tem muitos registos
- A coluna tem alta cardinalidade (muitos valores distintos)

Evitar quando:

- A tabela é pequena
- A coluna é atualizada com muita frequência
- A coluna tem baixa cardinalidade (ex: coluna active com só 0 e 1)

A Linguagem SQL

Prepared Statement

Um prepared statement é uma query SQL que é compilada uma vez pelo servidor e pode ser executada várias vezes com parâmetros diferentes.

É como um formulário com espaços em branco: a estrutura é sempre a mesma, só os valores mudam.

```
-- 1. Preparar
PREPARE nome_stmt FROM 'SELECT * FROM tabela WHERE coluna = ?';

-- 2. Definir parâmetro
SET @valor = 'exemplo';

-- 3. Executar
EXECUTE nome_stmt USING @valor;

-- 4. Libertar
DEALLOCATE PREPARE nome_stmt;
```

A Linguagem SQL

Prepared Statement

Para que servem?

Performance — a query é compilada/analisada só uma vez, mesmo executando várias vezes
Segurança — protegem contra SQL Injection, separando o código SQL dos dados
Reutilização — o mesmo statement pode ser executado com parâmetros diferentes

Exemplo

```
PREPARE stmt_aluguer FROM
    'SELECT first_name, last_name, email
     FROM customer
     WHERE store_id = ? AND active = ?';

SET @loja = 1;
SET @ativo = 1;
EXECUTE stmt_aluguer USING @loja, @ativo;

DEALLOCATE PREPARE stmt_aluguer;
```

A Linguagem SQL

Prepared Statement

Para que servem?

Performance — a query é compilada/analisada só uma vez, mesmo executando várias vezes
Segurança — protegem contra SQL Injection, separando o código SQL dos dados
Reutilização — o mesmo statement pode ser executado com parâmetros diferentes

Exemplo

```
PREPARE stmt_aluguer FROM
    'SELECT first_name, last_name, email
     FROM customer
     WHERE store_id = ? AND active = ?';

SET @loja = 1;
SET @ativo = 1;
EXECUTE stmt_aluguer USING @loja, @ativo;

DEALLOCATE PREPARE stmt_aluguer;
```

A Linguagem SQL

Procedimentos (Stored Procedures)

Um *stored procedure* é um **bloco de código SQL guardado no servidor** com um nome, que pode ser chamado sempre que necessário.

É como uma função numa linguagem de programação: define-se uma vez, chama-se várias vezes.

Sintaxe

```
DELIMITER $$

CREATE PROCEDURE nome_procedure()
BEGIN
    -- código SQL aqui
END$$

DELIMITER ;
```

Para chamar:

```
CALL nome_procedure();
```

A Linguagem SQL

Procedimentos (Stored Procedures)

Para que servem?

Reutilização — lógica definida uma vez, chamada em vários sítios

Performance — compilada e guardada no servidor

Segurança — o utilizador chama o *procedure* sem aceder diretamente às tabelas

Manutenção — alterações feitas num só lugar

A Linguagem SQL

Procedimentos (Stored Procedures)

Parâmetros: IN, OUT, INOUT

Tipo	Descrição
IN	Valor passado para a procedure (padrão)
OUT	Valor devolvido pelo *procedure*
INOUT	Passado e devolvido

```
CREATE PROCEDURE nome(IN param1 INT, OUT param2 VARCHAR(50))
```

A Linguagem SQL

Procedimentos (Stored Procedures)

Exemplos

Listar filmes por classificação:

```
DELIMITER $$
CREATE PROCEDURE sp_filmes_por_rating(
     IN p_rating VARCHAR(10)
)
BEGIN
     SELECT title, rental_rate, length
     FROM film WHERE rating = p_rating;
END$$
DELIMITER ;

CALL sp_filmes_por_rating('PG');
CALL sp_filmes_por_rating('R');
```

A Linguagem SQL

Procedimentos (Stored Procedures)

Exemplos

Contar o total de alugueres de um cliente:

```
DELIMITER $$

CREATE PROCEDURE sp_total_alugueres(
     IN p_customer_id        INT,
     OUT p_total             INT
)
BEGIN
     SELECT COUNT(*) INTO p_total
     FROM rental WHERE customer_id = p_customer_id;
END$$

DELIMITER ;

CALL sp_total_alugueres(1, @total);
SELECT @total;
```

A Linguagem SQL

Procedimentos (Stored Procedures)

Exemplos

Classificar cliente usando condições:

```
CALL sp_classificar_cliente(1);
```

```
DELIMITER $$

CREATE PROCEDURE sp_classificar_cliente(IN p_customer_id INT)
BEGIN
    DECLARE v_total INT;

    SELECT COUNT(*) INTO v_total
    FROM rental
    WHERE customer_id = p_customer_id;

    IF v_total > 30 THEN
        SELECT 'Cliente VIP' AS classificacao;
    ELSEIF v_total > 10 THEN
        SELECT 'Cliente Regular' AS classificacao;
    ELSE
        SELECT 'Cliente Ocasional' AS classificacao;
    END IF;
END$$

DELIMITER ;
```

A Linguagem SQL

Comparação

	VIEW	PREPARED STATEMENT	STORED PROCEDURE
Definição	Consulta guardada	Query pré-compilada	Programa armazenado no BD
Objetivo	Simplificar consultas	Melhorar performance e segurança	Automatizar lógica complexa
Reutilização	Sim	Sim (temporário)	Sim (persistente)
Lógica	Não suporta lógica	Muito limitada	Suporta lógica (IF, LOOP, etc.)
Parâmetros	Não	Sim	Sim
Performance	Média	Alta	Alta
Segurança	Controlo de acesso	Evita SQL Injection	Controlo avançado
Armazenamento	Permanente	Temporário (sessão)	Permanente
Execução	Automática ao fazer SELECT	EXECUTE após PREPARE	CALL nome_procedure()
Uso típico	Consultas simples	Queries repetidas	Processos complexos

Universidade do Minho

Ano Letivo: 2025/26

Bases de Dados

A Liguagem SQL (continuação)



António Abelha | abelha@di.uminho.pt